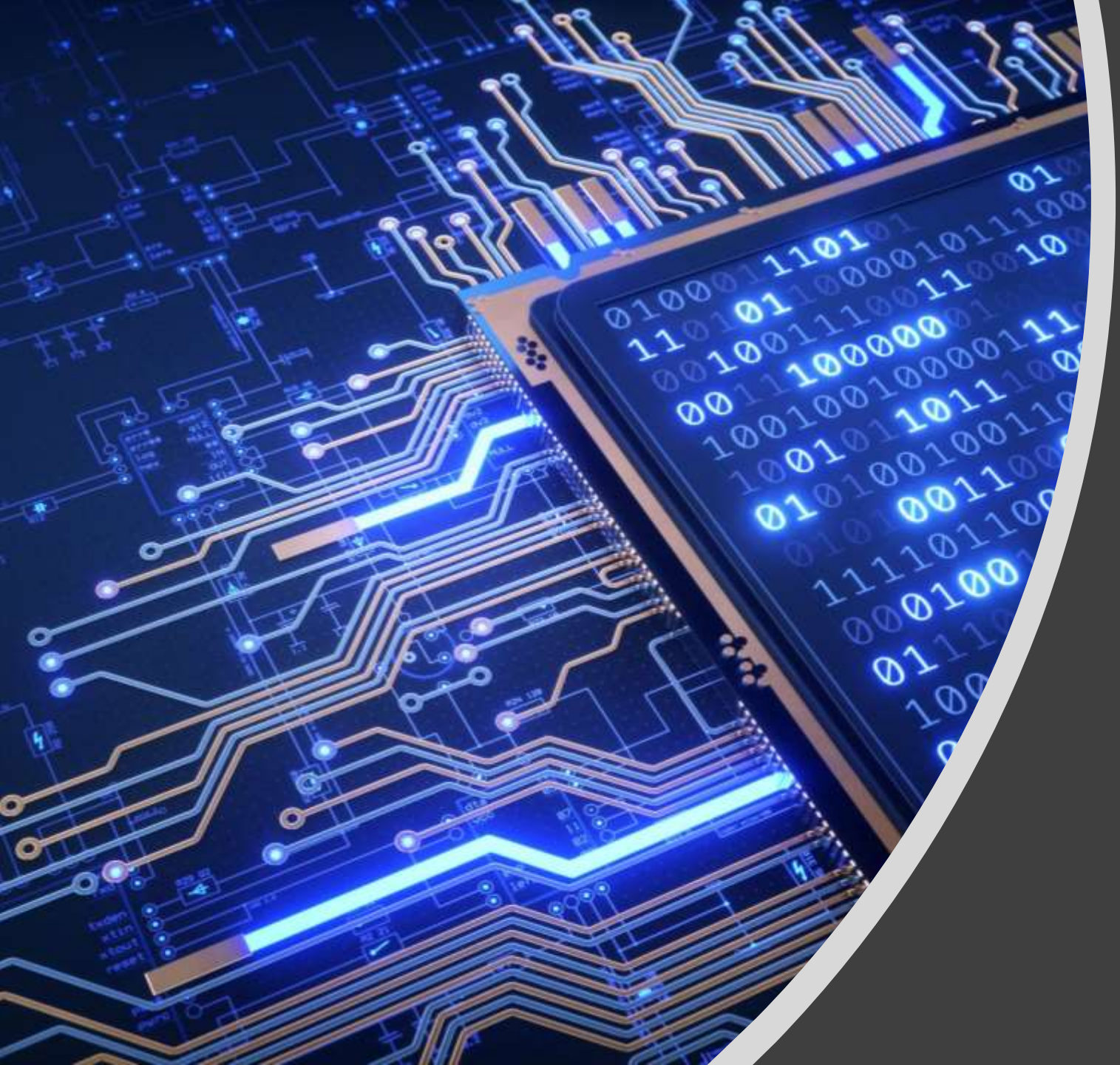


ERC20 Token

By Code Eater

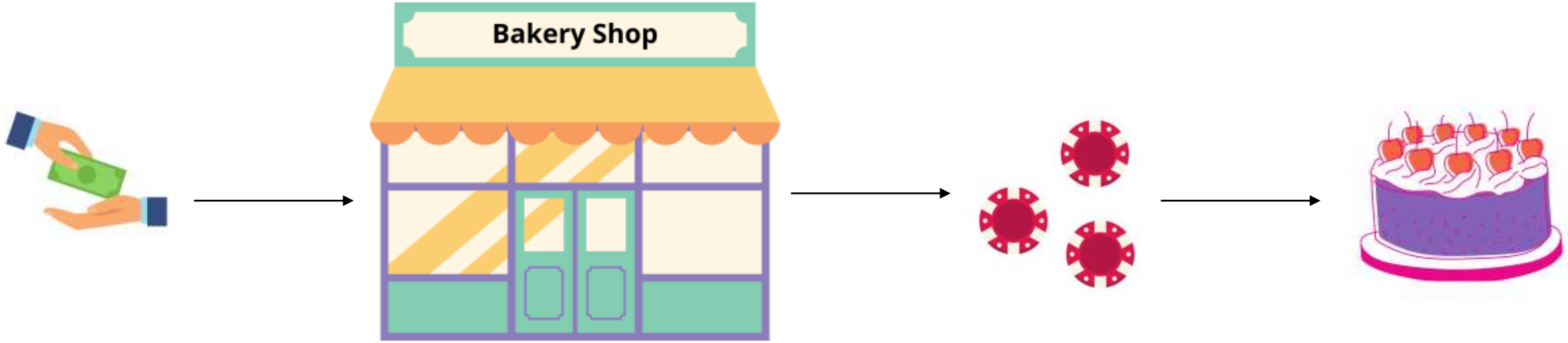
Cryptocurrencies

Coins

Tokens

Tokens

- **Tokens** are a type of **cryptocurrency** that represents an asset or specific use and resides on their **blockchain**.



ERC-20 Token

- ERC stands for Ethereum Request for Comments. An ERC is a form of proposal and its purpose is to define standards and practices. To know more – Click Here

- ERC20 is a proposal that intends **to standardize how a token contract** should be defined, how we interact with such a token contract and how these contracts interact with each other.

- ERC20 is a **standard interface** used by applications such as wallets, decentralized exchanges, and so on to interact with tokens.

- A full compatible ERC20 Token must implement **6 functions and 2 events**.

IERC-20 Interface

- IERC20 Interface – Click Here

Mapping Data Structure

- mapping(address=>mapping(address=>uint)) allowed;

	address A	address B	address C
address 1		100	
address 2			
address 3			

allowed[address 1][address B] =100

Note – Remember this is not an array. But you can imagine it like a 2D array.

Code Eater

Mapping Data Structure

- mapping(address=>mapping(address=>uint)) allowedTokens;

```
allowedTokens[address 1][address A]>=100
```

	address A	address B	address C
address 1	100		
address 2			200
address 3			

allowed[address 2][address C] =200

Code Eater

Openzeppelin

- OpenZeppelin is **an open-source framework to build secure smart contracts**.
- Click Here - Openzeppelin
- Click Here – Openzeppelin ERC20

Decimal Concept

- The EVM does not support decimal numbers (to prevent rounding errors related to the network nodes running on different architectures), so all numbers are integers.

- The ERC-20 token standard defines the `decimals()` function that your contract implements, effectively shifting all numbers by the declared number of zeros to simulate decimals.

- So if you wanted to mint 1 token with 18 decimals, you'd need to pass the value `1000000000000000000` (18 zeros).

- In your case, 1 million tokens (6 zeros) with 18 decimals is represented as `1000000000000000000000000` (24 zeros).

- Real Total Supply = Initial Supply Supply * 10^-Decimals (In ERC20 token contract from openzeppelin it is 10^-18)

Code Eater

Decimal Concept

The decimals property is typically set when the token is created and indicates how divisible the token is. The value of decimals determines how many zeros come after the decimal point.

Example
Let's consider a token with decimals set to 18, which is a common setting for ERC-20 tokens. This means the token has 18 decimal places.

Token with 18 Decimals
- **Token Symbol:** TKN

1 TKN can be divided into 10^18 smallest units. Thus, the smallest unit of TKN is 0.000000000000000001 TKN (also known as a "wei" in Ethereum).

Calculating Total Supply
If a token contract has a totalSupply of 1,000,000 TKN and decimals set to 18, it means the totalSupply property in the smart contract would actually be:

Decimal Concept

totalSupply = 1,000,000 * 10^18

This value is stored as an integer in the contract:

totalSupply = 1,000,000,000,000,000,000,000,000

ChatGpt Response for decimals – Click Here

Decimal Concept

Sending Tokens:

When transferring tokens, you also need to account for the decimals. For instance, if you want to transfer 1.5 TKN, you actually need to specify:

amount = 1.5 * 10^18 = 1,500,000,000,000,000,000

Thank You

Instagram - @codeeater21
YouTube - @codeeate21

```
// 🔹 Difference Between Token and Coin in Crypto
// Definition:
// Coin: A cryptocurrency that operates on its own blockchain.
// Token: A digital asset that operates on an existing blockchain.


// Blockchain:
// Coin: Has its own independent blockchain.
// Token: Uses an existing blockchain (like Ethereum, Binance Smart Chain,
etc.).


// Purpose:
// Coin: Used mainly as currency (store of value, medium of exchange).
// Token: Used for utility (access to services, governance, NFTs, etc.).


// Examples:
// Coin: Bitcoin (BTC), Ethereum (ETH), Binance Coin (BNB).
// Token: USDT (Tether), Uniswap (UNI), Chainlink (LINK).

// Transaction Fees:
// Coin: Pays network fees in its own native coin.
// Token: Pays fees in the native coin of the blockchain (e.g., ERC-20
tokens pay gas fees in ETH).


// Creation:
// Coin: Requires a separate blockchain to be built.
// Token: Created using smart contracts on an existing blockchain.\


// 🔹 Explanation with Example
// 1⃣ Coin Example: Bitcoin (BTC)
// Bitcoin has its own blockchain and is mainly used for transactions and
store of value.
// Mining and validation occur within the Bitcoin network.
// Transaction fees are paid in BTC.
// 2⃣ Token Example: USDT (Tether) on Ethereum
// USDT is a stablecoin that runs on Ethereum's blockchain (ERC-20 token).
```

```
// Transactions require ETH for gas fees, not USDT itself.
// It does not have its own blockchain.


// 🔹 Simple Analogy
// Coin = A country's currency (e.g., USD, INR) → Has its own financial
system.
// Token = A gift card (e.g., Amazon Gift Card) → Runs on an existing
system but has a specific use.
```

```
// SPDX-License-Identifier: MIT
//this is just for show how ERC20 looks. it.in the next code actual ERC20
is there.
pragma solidity ^0.8.20;

interface IERC20 {
    event Transfer(address indexed from, address indexed to, uint256
value);
    event Approval(
        address indexed owner,
        address indexed spender,
        uint256 value
    );

    function totalSupply() external view returns (uint256);

    function balanceOf(address account) external view returns (uint256);

    function transfer(address to, uint256 value) external returns (bool);

    function allowance(address owner, address spender)
        external
        view
        returns (uint256);

    function approve(address spender, uint256 value) external returns
(bool);

    function transferFrom(
        address from,
```

```
        address to,
        uint256 value
    ) external returns (bool);
}

contract ERC20 is IERC20 {
    uint256 public override totalSupply = 1000;
    uint8 public decimals = 0;
    string public name = "TestToken";
    string public symbol = "TTK";

    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowances;

    constructor() {
        balanceOf[msg.sender] = totalSupply;
    }

    function getBalance(address account) external view returns (uint256) {
        return balanceOf[account];
    }

    function transfer(address to, uint256 value)
        external
        override
        returns (bool)
    {
        require(balanceOf[msg.sender] >= value, "Insufficient balance");
        require(to != address(0), "Invalid address");

        balanceOf[msg.sender] -= value;
        balanceOf[to] += value;

        emit Transfer(msg.sender, to, value);
        return true;
    }

    //writing a cheque
    function approve(address spender, uint256 value)
        external
```

```
        override
        returns (bool)
    {
        require(spender != address(0), "Invalid address");

        allowances[msg.sender][spender] = value;

        emit Approval(msg.sender, spender, value);
        return true;
    }

    //reading a cheque
    function allowance(address owner, address spender)
        external
        view
        override
        returns (uint256)
    {
        return allowances[owner][spender];
    }

    function transferFrom(
        address from,
        address to,
        uint256 value
    ) external override returns (bool) {
        require(allowances[from][msg.sender] >= value, "Allowance
exceeded");
        require(balanceOf[from] >= value, "Insufficient balance");
        require(to != address(0), "Invalid address");

        allowances[from][msg.sender] -= value;
        balanceOf[from] -= value;
        balanceOf[to] += value;

        emit Transfer(from, to, value);
        return true;
    }

}
```

This is the actual openzeppelin ERC-20 token code.and here transferFrom is not working correctly.

```
// contracts/GLDToken.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract CTEToken is ERC20 {
    constructor(uint256 initialSupply) ERC20("Silver", "Deci1") {
        _mint(msg.sender, initialSupply);
    }

    function decimals() public pure override returns (uint8) {
        return 18;
    }
}
```